INSTITUTO FEDERAL
RIO GRANDE DO SUL

Estrutura de Dados II

Algoritmos de Ordenação – Parte I

- Ordenação por Bolha (Bubble Sort)
- Ordenação por Seleção (Select Sort)
- Ordenação por Inserção (Insert Sort)

Revisando

- **Pesquisa Sequencial** compara cada item de um conjunto de elementos.
- **Pesquisa binária** verifica se o elemento está no centro, se está na parte inferior ou na parte superior no conjunto de elementos.
- **Na pesquisa binária**, a cada execução é eliminado a metade do conjunto até finalizar o conjunto inteiro.

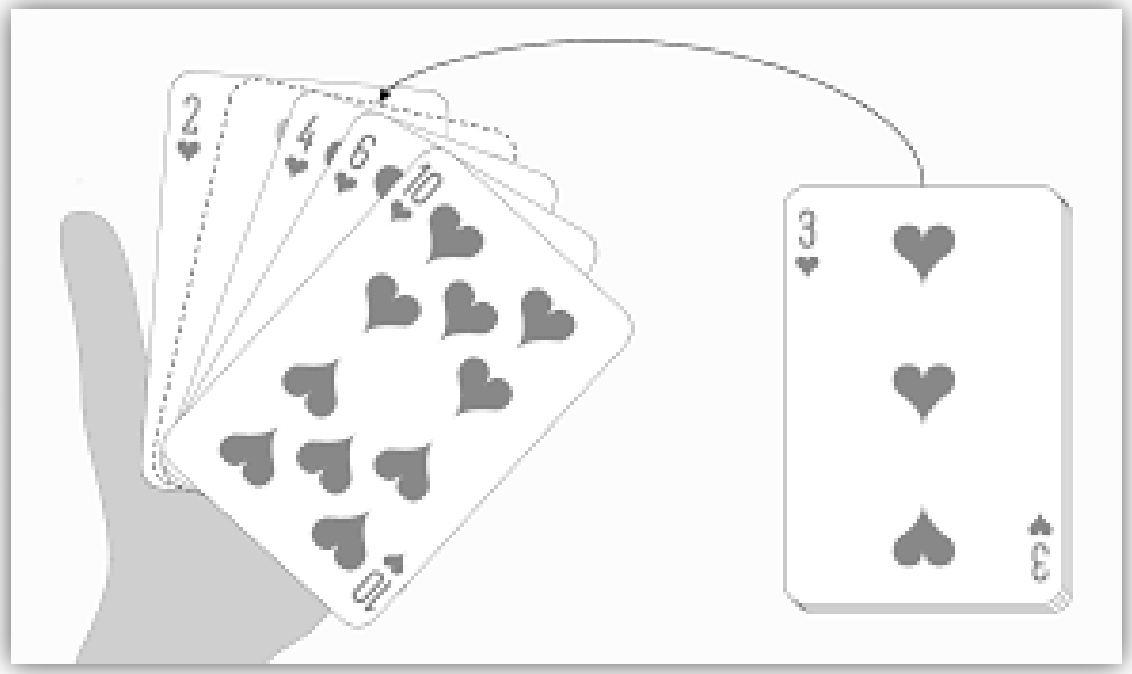
INSTITUTO FEDERAL
RIO GRANDE DO SUL

Revisando

- **“Pesquisar”** é (deve ser) uma operação rotineira em um conjunto de dados.
- **Pesquisas eficientes** exigem **Dados Ordenados** (ou estruturas ordenadas, índices por exemplo).
- **Ordenar Dados exige processamento extra.**

Ordenação de Dados

Algoritmos de Ordenação de Dados



INSTITUTO FEDERAL
RIO GRANDE DO SUL

Algoritmos de Ordenação

- Ordenação de Dados é um processo que coloca um conjunto de dados em uma certa ordem, ou seja, realiza a ordenação (classificação) dos dados.

- Durante a ordenação de dados podem ser usadas as o formato numérico ou alfabético (ou alfanumerico) como critério de ordenação. Pode também ser usado o formato Data (temporal) ou outros atributos definidos (ex. tamanho, etc)

Algoritmos de Ordenação

Vantagens:

- Um conjunto de dados ordenados possibilita o acesso dos seus dados de modo mais eficiente (velocidade).

Desvantagens:

- Requer **processamento extra para ordenação** e criação de mecanismos e algumas vezes mecanismos externos (índices) para manter a ordenação.

Algoritmos de Ordenação

O que diferencia um algoritmo do outro?

Basicamente e Eficiência: Tempo que leva para ordenar os dados, os recursos gastos (processamento) e o tipo de dado.

Os algoritmos são classificados pela sua complexidade:

*** O(n) – n vezes**

*** $O(n^2)$ – n * n vezes (tempo quadrático)**

*** (log n) logarítmica (diminuem a cada instância executada)**

Algoritmos de Ordenação

Estáveis x Não Estáveis

Um algoritmo de ordenação é considerado estável quando consegue preservar a ordem de registro de chaves iguais.

3-a	2-c	2-d	1-e	4-f

Obrigatoriamente o resultado será:

1-e	2-c	2-d	3-a	4-f

Os algoritmos não estáveis sujeitam os elementos associados aos objetos a serem ordenados:

1-e	2-d	2-c	3-a	4-b

INSTITUTO FEDERAL
RIO GRANDE DO SUL

Algoritmos de Ordenação

Ordenação Interna x Ordenação Externa

Ordenação Interna:

Todos os elementos a serem ordenados cabem na memória. Os registros podem ser acessados diretamente.

Ordenação Externa:

Os elementos não cabem na memória principal e os registros são acessados sequencialmente ou em blocos.

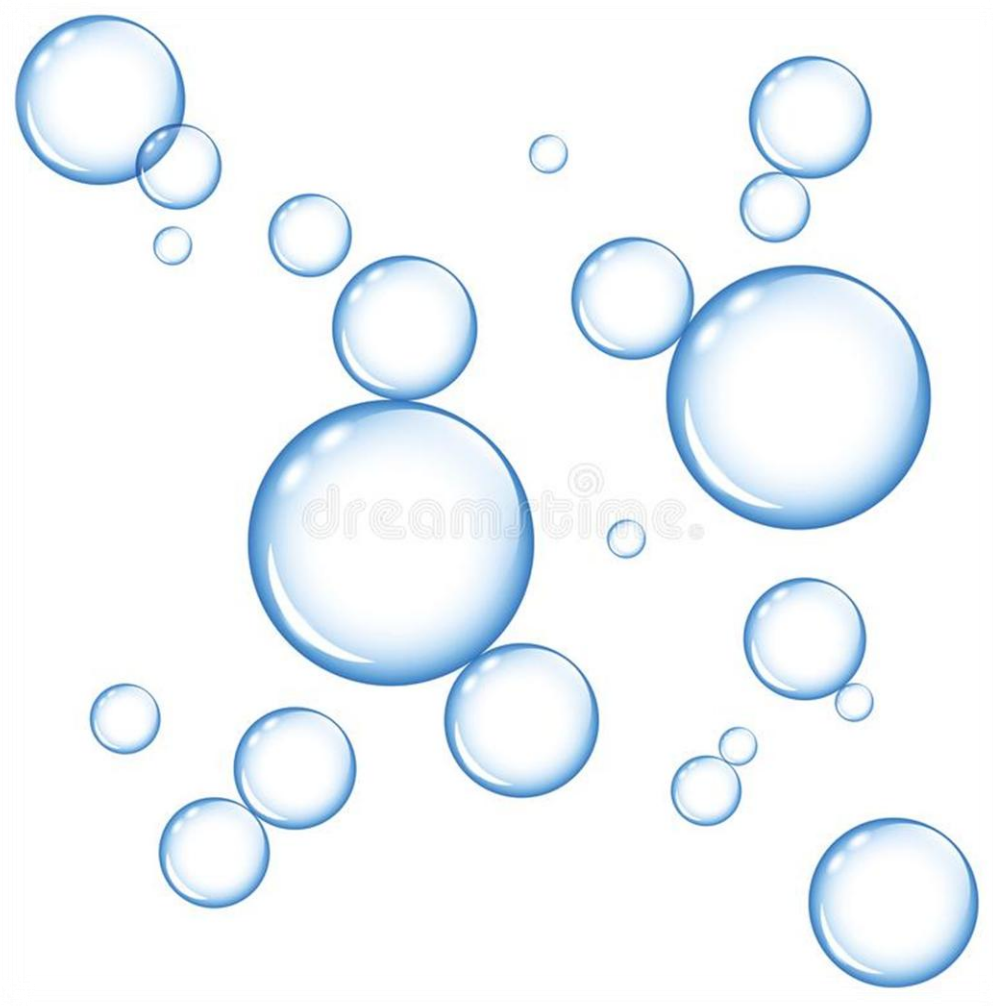
INSTITUTO FEDERAL
RIO GRANDE DO SUL

Algoritmos de Ordenação

Alguns Algoritmos de Ordenação:

- Bubblesort
- Ordenação por Contagem
- Ordenação por Inserção
- Mergesort
- Quicksort
- Ordenação por Seleção

ORDENAÇÃO POR BOLHAS



INSTITUTO FEDERAL
RIO GRANDE DO SUL

Algoritmo de Ordenação Bubblesort

Algoritmo O ***bubble sort***, ou ordenação por flutuação (literalmente "por bolha") é um dos algoritmos de ordenação mais simples.

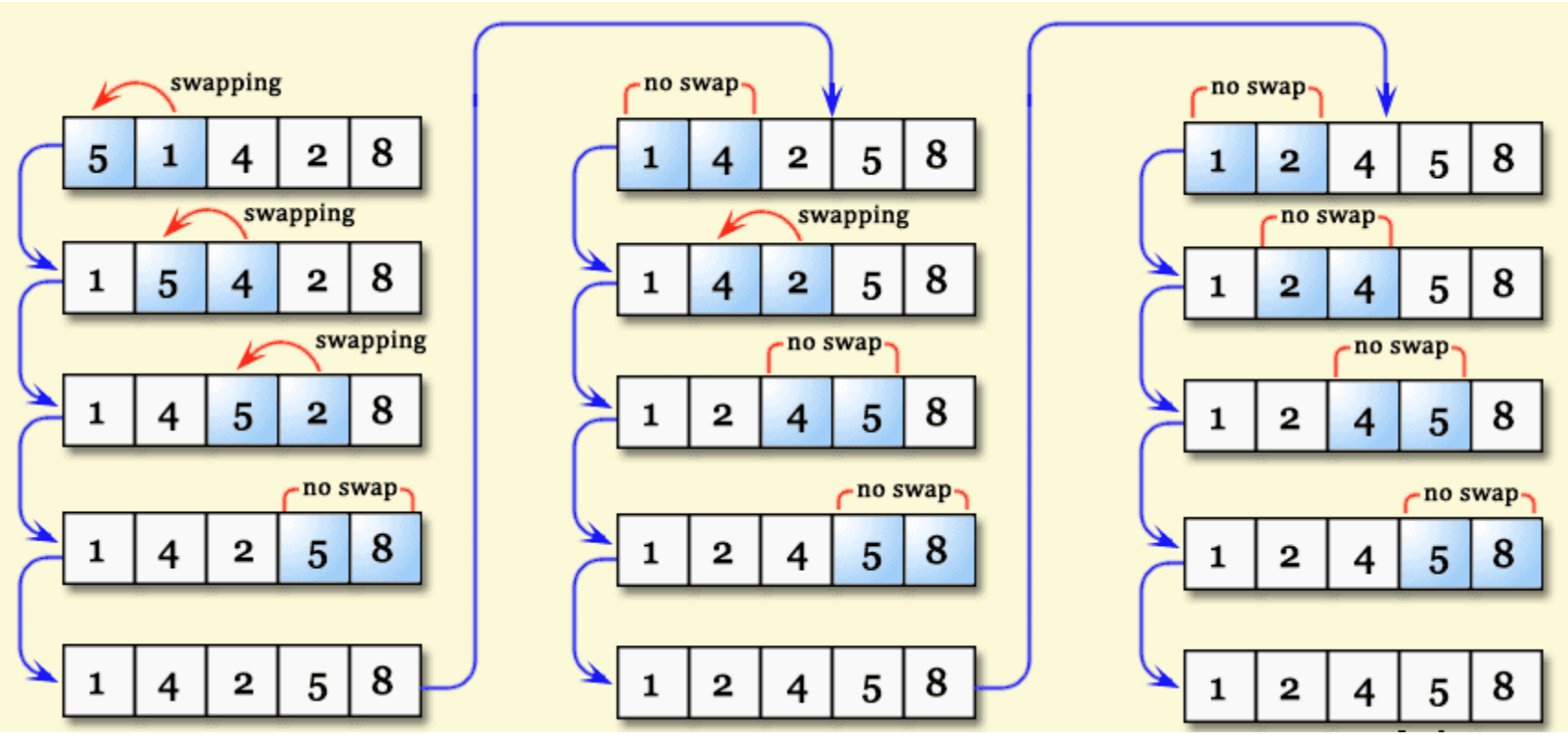
Em uma estrutura de dados desordenada inicia-se o algoritmo pelo primeiro elemento, depois faz-se a comparação dele com todos os que estão depois dele na estrutura desordenada.

Algoritmo de Ordenação Bubblesort

A ideia é percorrer o vetor diversas vezes, a cada passagem fazendo flutuar para o topo o maior elemento da sequência.

Essa movimentação lembra a forma como as bolhas em um tanque de água procuram seu próprio nível, e disso vem o nome do algoritmo.

Bubblesort


swapping
5 1 4 2 8
swapping
1 5 4 2 8
swapping
1 4 5 2 8
no swap
1 4 2 5 8
1 4 2 5 8
no swap
1 4 2 5 8
swapping
1 4 2 5 8
no swap
1 2 4 5 8
no swap
1 2 4 5 8
1 2 4 5 8
no swap
1 2 4 5 8
no swap
1 2 4 5 8
no swap
1 2 4 5 8
1 2 4 5 8

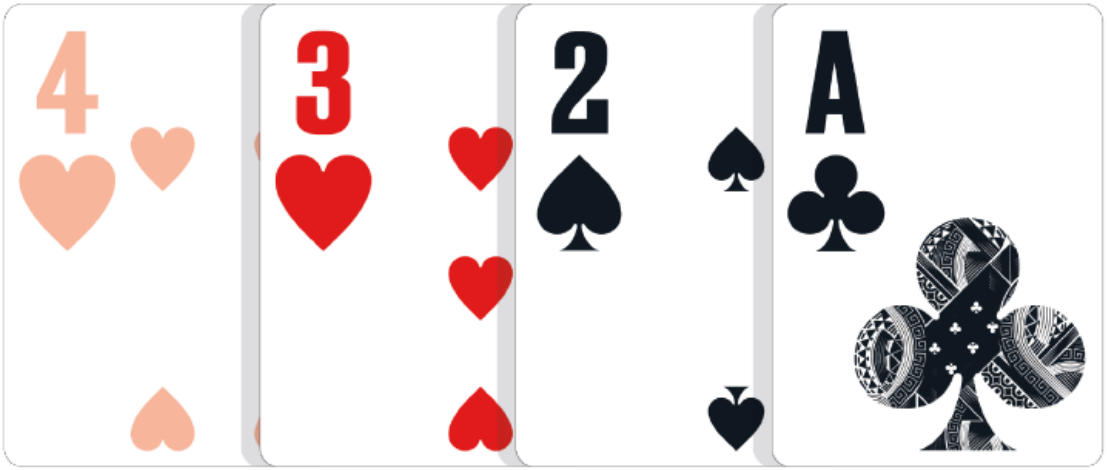
Algoritmo de Ordenação Bubblesort

```
#include <stdio.h>
#include <stdlib.h>

void  main ()
{
//Inicializaçã do Vetor
    int v[5]= {1,7,4,3,5}, n=5;
          int i, j = 0,  aux;

//Algoritmo de Ordenação
          while (j < n)
          {
                    for(i = 0; i < n-1; i++)
                              if(v[i] > v[i + 1])
                              {
                                        aux=v[i];
                                        v[i] = v[i+1];
                                        v[i+1] = aux;
                              }
                    j++;}
//Laço de impressão do Vetor
          for (int q=0; q<5; q++)
          {
        printf("%d \n",v[q]);
    }
    system("pause");  }
```

ORDENAÇÃO POR SELEÇÃO



INSTITUTO FEDERAL
RIO GRANDE DO SUL

Algoritmo de Ordenação por Seleção

O *selection sort* compara a cada interação um elemento com os outros, visando encontrar o menor.

Dessa forma, podemos entender que não existe um melhor caso mesmo que o vetor esteja ordenado ou em ordem inversa serão executados.

A complexidade deste algoritmo será sempre $O(n^2)$.

INSTITUTO FEDERAL
RIO GRANDE DO SUL

Algoritmo de Ordenação por Seleção

Seu funcionamento consiste em se passar sempre o menor valor do conjunto para a primeira posição (ou o maior dependendo da ordem requerida), depois o de segundo menor valor para a segunda posição, e assim é feito sucessivamente com os (n-1) elementos restantes, até os últimos dois elementos.

Ele é um algoritmo simples de ser implementado em comparação aos demais (sem recursividade).

Não necessita de um vetor auxiliar, ocupa menos memória.

Ele é uns dos mais velozes na ordenação para um conjunto de dados pequenos.

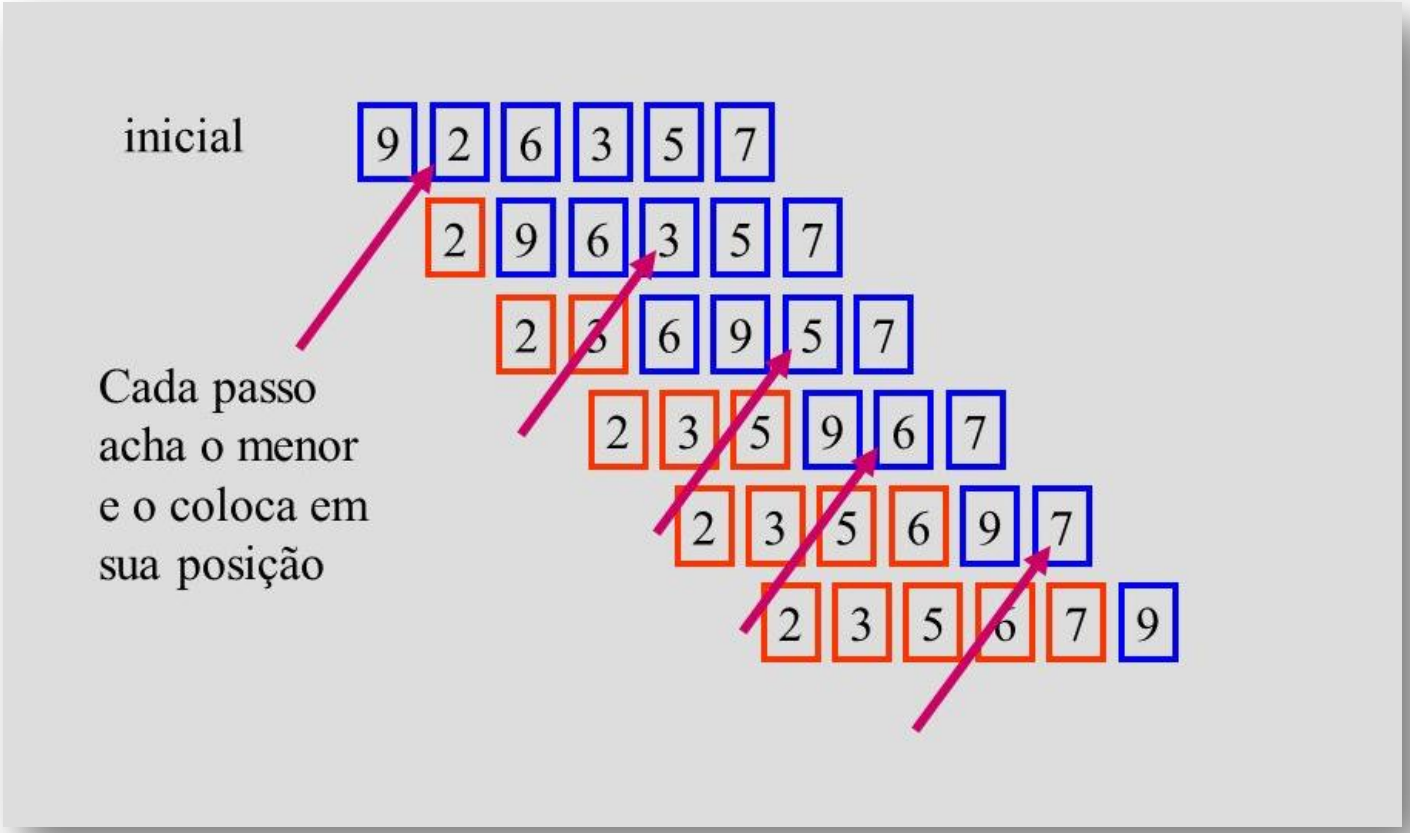
Algoritmo de Ordenação por Seleção

Ou Seja, por trás do algoritmo de ordenação por seleção é o executado para um Conjunto de N números, os seguintes passos:

a) A cada iteração do algoritmo, selecione o menor elemento do conjunto deixe na sua posição correta final.

b) Resolva o problema para os elementos restantes.

Ordenação por Seleção



Algoritmo de Ordenação por Seleção

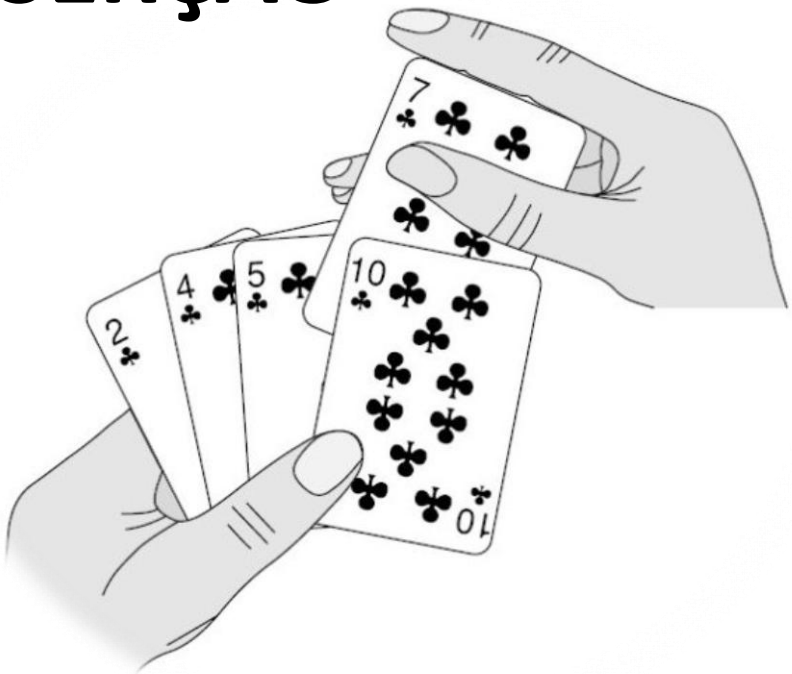
```
...

main () {
 //Inicialização do Vetor
    int num[5]= {1,7,4,3,5}, tam=5;

//Algoritmo de Ordenação
  int i, j, min;
  for (i = 0; i < (tam-1); i++) {
    min = i;
    for (j = (i+1); j < tam; j++) {
      if(num[j] < num[min]) {
        min = j;
      }}
    if (i != min) {
      int swap = num[i];
      num[i] = num[min];
      num[min] = swap;}}

//Laço de impressão do Vetor
for (int q=0; q<5; q++)
{
        printf("%d \n",num[q]);    }
}
```

ORDENAÇÃO POR INSERÇÃO



Ordenação por Inserção

É um método que percorre um vetor de elementos da esquerda para a direita e à medida que avança vai ordenando os elementos à esquerda.

Possui complexidade O(n) no melhor caso e $O(n^2)$ no caso médio e pior caso.

O funcionamento do algoritmo consiste em cada passo a partir do segundo elemento selecionar o próximo item da sequência e colocá-lo no local exato de acordo com o critério de ordenação.

INSTITUTO FEDERAL
RIO GRANDE DO SUL

Algoritmo de Ordenação por Inserção

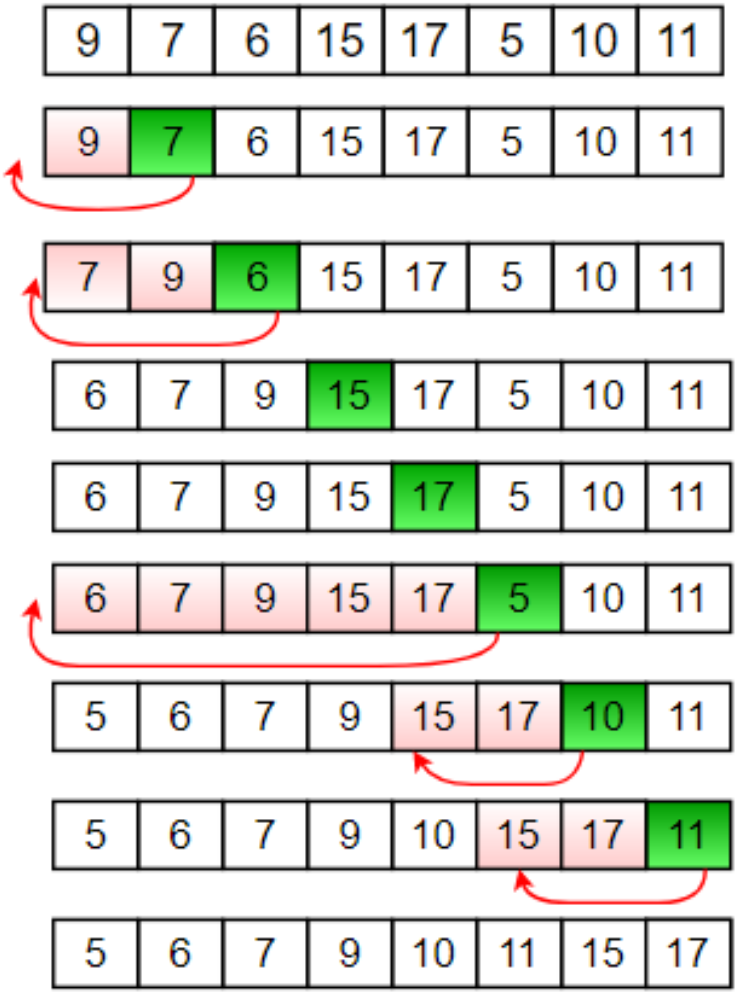
É um algoritmo simples, eficiente quando aplicado a um pequeno número de elementos.

Seu funcionamento é de percorrer um vetor de elementos da esquerda para a direita e à medida que avança vai deixando os elementos mais à esquerda ordenados.

Algoritmo de Ordenação por Inserção

- É possível comparar o *Insertion Sort* com o modo como de organizar cartas de um baralho.
- Imagine as cartas na mão e elas estão ordenadas. Você recebe uma nova carta, deve inserir na posição correta das cartas, de forma a que as cartas obedeçam à ordenação.
- A cada nova carta adicionada, a nova carta pode ser menor que algumas das cartas que você já tem na mão ou maior, e assim, você começa a comparar a nova carta com todas as cartas na sua mão até encontrar sua posição correta.
- Você insere a nova carta na posição correta. Então, você recebe outra carta e repete o mesmo procedimento, sucessivamente até a última.

Ordenação por Inserção



Algoritmo de Ordenação por Inserção

```
#include <stdio.h>
#include <stdlib.h>

main ()  {
    //Inicializaçã do Vetor
    int v[5]= {1,7,4,3,5} n=5;
    int i, j, chave;

    //Algoritmo de Ordenação
        for(j=1; j<n; j++)
        {
                chave = v[j];
                i = j-1;
                while(i >= 0 && v[i] > chave) {
                        v[i+1] = v[i];
                        i--;}
                        v[i+1] = chave;}

    //Laço de impressão do Vetor
        for (int q=0; q<5; q++)   {
                printf("%d \n",v[q]);
        }
}
```

Laboratório

- Execute o Programa “Gera_RAND” para criar um arquivo com 100.000 números randômicos.

- Altere os programas do BubbleSort, SelectSort e InsertSort para que leia o arquivo e gere um arquivo ordenado.

- **Execute os três algoritmos de ordenação fornecidos e registre o tempo de execução de cada um.**
  - Realize os testes com dois tipos de entrada:
  - Um arquivo contendo dados gerados aleatoriamente.
  - Um arquivo contendo os mesmos dados, porém ordenados de forma decrescente (caso estejam originalmente em ordem crescente, ou vice-versa).

- Com base nos resultados, responda: **qual dos três algoritmos apresentou o melhor desempenho em termos de tempo de execução?**

INSTITUTO FEDERAL
RIO GRANDE DO SUL

```c
#include <time.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
main ()
{
    //Inicializaçã do Vetor
    FILE *txt;
    txt = fopen("Arquivo.txt", "r");
    float v[100000], n=100000;
    float j = 0,  aux;
    long int  i=0;

    printf("\nCarregando o Arquivo no Vetor");
    while (i< n)
    {
        fscanf(txt, "%f",&v[i]);
        i++;
    }
    fclose(txt);

    //Algoritmo de Ordenação
    //aqui

    printf("\nImprimindo Vetor Ordenado");
    txt = fopen("Arquivo_ordenado.txt", "w");

    //Laço de impressão do Vetor
    for (i=0; i<n; i++)
    {
        fprintf(txt, "%.0f\n",v[i]);
    }
    fclose(txt);
}
```